

DevOps Interview Questions (Easy to Advanced)

EASY (Foundational)

1. What are the typical CI/CD stages?

Source → Build → Test → Package → Security scan → Deploy → Verify → Rollback (if needed).

2. What's the difference between CI, CD, and Continuous Deployment?

CI: Merge and test code frequently.

Continuous Delivery: Always releasable builds; manual prod trigger.

Continuous Deployment: Auto-deploy to production after successful tests.

3. Name three CI/CD tools and one key difference.

Jenkins, GitHub Actions, GitLab CI. Jenkins is server-managed; GA/GitLab are repo-native and YAML-based.

4. What is Infrastructure as Code (IaC) and why is it used?

Managing infrastructure via code for repeatability, version control, automation, and drift prevention.

5. Terraform vs Ansible — what's the difference?

Terraform is declarative and provisions infra; Ansible is procedural and configures systems. They're often used together.

6. How do containers differ from virtual machines?

Containers share the host OS kernel (lightweight); VMs virtualize hardware (heavier but isolated).

7. What is Kubernetes used for?

For orchestrating, scaling, and managing containerized workloads declaratively.

8. What's the difference between a Kubernetes Deployment and a Service?

Deployment manages pod rollouts; Service provides stable networking and load balancing.

9. What are SLAs, SLOs, and SLIs?

SLA: Business agreement (e.g., 99.9% uptime).

SLO: Internal reliability goal.

SLI: Actual measured metric (latency/error rate).

10. Monitoring vs Logging vs Tracing — tools?

Monitoring: Prometheus, Grafana.

Logging: ELK/EFK/Loki.

Tracing: Jaeger, OpenTelemetry.

11. How does auto-scaling improve reliability?

Automatically adjusts capacity based on metrics to maintain performance under load.

12. What is GitOps?

Declarative infrastructure and apps managed via Git as the single source of truth, synced by tools like ArgoCD or Flux.

13. What are Prometheus and Grafana used for?

Prometheus collects metrics; Grafana visualizes and alerts on them.

14. What's the role of Helm in Kubernetes?

Helm is a package manager for Kubernetes, simplifying app deployments via reusable charts.

15. Give an example of a rollback strategy.

Blue-green deployment switch, canary rollback based on metrics, or kubectl rollout undo.

MEDIUM (Practical/Scenario)

21. How would you design a CI/CD pipeline using Jenkins or GitHub Actions?

Trigger on PR → build → test → scan → build container → push to registry → deploy via Helm → post-deploy checks.

22. How do you ensure a Jenkins pipeline fails fast?

Use set -e, unit tests early, parallelize slow stages, and enforce code-quality gates.

23. How do you manage secrets in CI/CD?

Use HashiCorp Vault, AWS Secrets Manager, or sealed secrets; never store plaintext in repos.

24. How does Terraform manage state?

Stores real resource mappings in a state file; use remote backends with locking to avoid conflicts.

25. How do you debug a pod in CrashLoopBackOff?

Check logs and events, validate configs, review probes, resource limits, and container commands.

26. How can you minimize downtime during K8s upgrades?

Use surge updates, PDBs, canary nodes, version skew compliance, and pre-upgrade testing.

27. How do you handle high latency in production?

Check metrics (latency, CPU, DB queries), isolate the layer, add caching, or scale horizontally.

28. What are best practices for alert thresholds?

Alert on user-impacting SLIs, use multi-window burn rates, and avoid false positives.

29. What are the different types of deployment strategies, Blue-Green vs Canary deployments?

Blue-Green swaps entire environments; Canary gradually releases changes to small user subsets.

30. Describe your RCA process after an outage.

Document timeline, analyze root cause, apply fix, identify prevention actions—blamelessly.

31. How is ArgoCD used for deployments?

Syncs desired state from Git to clusters automatically with rollback and health checks.

22. How do you ensure compliance in CI/CD pipelines?

Shift-left security: run SAST, DAST, dependency scans, enforce OPA/Conftest policies, require approvals.

33. How do you optimize cloud costs while maintaining performance?

Right-size instances, enable autoscaling, use spot/preemptible VMs, cache content, and monitor spend.

34. How do you set up disaster recovery for cloud-native systems?

Define RPO/RTO, back up infra and data across regions, automate failover with IaC and health checks.

35. How would you design a scalable, HA (AWS/GCP/Azure) architecture using Terraform?

Multi-AZ VPC, ALB → ASG/EKS, RDS Multi-AZ, ElastiCache, S3, CloudFront, Route53 failover; codified via Terraform modules.

ADVANCED (Strategic/Real-World)

36. How do you achieve zero-downtime deployments in Kubernetes?

Rolling updates with readiness probes, canary rollouts, feature flags, and schema-compatible DB migrations.

37. How do you measure and improve MTTR(Mean time to repair)?

Enhance observability, define clear runbooks, automate rollbacks, and conduct blameless postmortems.

38. How can rollback be automated on failed health checks?

Use metrics-based rollback in Argo Rollouts/Flagger or deployment thresholds in CI/CD.

39. How do you enforce security across multiple clusters?

Centralize policy enforcement via OPA Gatekeeper/Kyverno, enable PodSecurityStandards, and sign images.

40. How do you design cross-region failover for critical workloads?

Multi-region active-active setup, DNS failover, replicated databases, and tested failover plans.

41. How do you reduce on-call fatigue?

Use SLO-based alerts, deduplicate notifications, auto-remediate known issues, and rotate schedules fairly.

42. How do you implement chaos engineering?

Inject controlled failures, observe impact, validate recovery, and harden systems iteratively.

43. How do you manage SLAs/SLOs across microservices?

Define SLIs (latency, error rate), set budgets, automate monitoring, and gate releases on error budgets.

44. How do you migrate a monolith to microservices with DevOps practices?

Use the Strangler pattern, containerize services, adopt CI/CD, establish service mesh & observability.

45. How do you ensure governance in large Terraform environments?

Use versioned modules, policy-as-code (OPA), code reviews, remote state locking, and audit logs.

46. How do you integrate DevSecOps into CI/CD?

Add SAST, DAST, IaC scans, SBOM generation, and image signing—enforce all as pipeline gates.

47. How does AIOps enhance observability and incident response?

AIOps platforms (like Dynatrace, Moogsoft, Datadog AI, or Elastic AIOps) apply machine learning to correlate metrics, logs, and traces — automatically detecting anomalies and suppressing alert noise. This shortens MTTR by identifying root causes faster and recommending fixes based on historical patterns.

48. How would you implement an AIOps workflow in an existing DevOps ecosystem?

Integrate your telemetry stack (Prometheus, ELK, OpenTelemetry) with an AIOps layer that:

1. Ingests data streams in real time,
2. Applies anomaly detection and trend analysis,
3. Triggers automated remediation through runbooks (Ansible/Terraform), and
4. Feeds insights back into CI/CD or SRE dashboards.

This bridges observability and automation using data-driven intelligence.

49. What challenges arise when adopting AIOps at scale, and how do you mitigate them?

Common challenges include:

- **Data quality & noise** → standardize telemetry formats and tagging.
- **Model drift** → retrain ML models on recent production data.
- **Trust & transparency** → surface explainable insights, not black-box outputs.
- **Integration complexity** → use APIs to embed AIOps with existing tools (Jenkins, ServiceNow, PagerDuty).

Success depends on gradual rollout, feedback loops, and tight SRE alignment.

50. How does MLOps differ from traditional DevOps, and what additional components (like data pipelines, model monitoring, and retraining) does it introduce to the workflow?

MLOps extends traditional DevOps by adding the data and model lifecycle on top of code delivery — so you're not just deploying applications, but continuously training, validating, and deploying ML models.

It introduces components like data pipelines, feature stores, model versioning, monitoring, and automated retraining to keep models accurate as data changes.

Hope this helps!

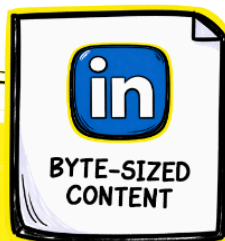
-V

YOU MIGHT ALSO LIKE 



Thanks for following!

Your Go-To Cloud & DevOps Girl ☁️



Click for more!

A black curved arrow pointing from the circular profile picture towards the 'Click for more!' text.